

```

// IntArrayDescSorter.h
#ifndef INT_ARRAY_DESC_SORTER_H
#define INT_ARRAY_DESC_SORTER_H

class IntArrayDescSorter
{
public:
    // Overload the function call operator.
    // Usage: sorter(array, n);
    // It sorts the first n elements of array in descending order.
    void operator()(int array[], int n) const;
};

#endif // INT_ARRAY_DESC_SORTER_H

```

```

.....

// IntArrayDescSorter.cpp
#include "IntArrayDescSorter.h"
#include <algorithm>
#include <functional>

void IntArrayDescSorter::operator()(int array[], int n) const
{
    // If the pointer is null or the array has 0 or 1 element,
    // no sorting is necessary.
    if (array == nullptr || n <= 1)
    {
        return;
    }

    // Sort array[0] to array[n - 1] from large to small.
    std::sort(array, array + n, std::greater<int>());
}

```

```

.....

// main.cpp
#include <iostream>
#include "IntArrayDescSorter.h"

int main()
{
    int array[] = {5, 1, 9, 3, 7, 2};
    int N = sizeof(array) / sizeof(array[0]);

    IntArrayDescSorter obj;
    obj(array, N);    // Equivalent to obj.operator()(array, N)

    for (int i = 0; i < N; ++i)
    {
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}

```